

**Минобрнауки России**

**Юго-Западный государственный университет**

Кафедра программной инженерии

**ОТЧЕТ**

о преддипломной (производственной) практике

наименование вида и типа практики

на (в) **ООО «Предприятие ВТИ-Сервис»**

наименование предприятия, организации, учреждения

Студента **4 курса, группы ПО-226**

курса, группы

**Комиссарова Богдана Александровича**

фамилия, имя, отчество

Руководитель практики от  
предприятия, организации,  
учреждения

Оценка \_\_\_\_\_

\_\_\_\_\_  
должность, звание, степень

\_\_\_\_\_  
фамилия и. о.

\_\_\_\_\_  
подпись, дата

Руководитель практики от  
университета

Оценка \_\_\_\_\_

**К.Т.Н. доцент**

\_\_\_\_\_  
должность, звание, степень

**Чаплыгин А. А.**

\_\_\_\_\_  
фамилия и. о.

\_\_\_\_\_  
подпись, дата

**Члены комиссии**

\_\_\_\_\_  
подпись, дата

\_\_\_\_\_  
фамилия и. о.

\_\_\_\_\_  
подпись, дата

\_\_\_\_\_  
фамилия и. о.

\_\_\_\_\_  
подпись, дата

\_\_\_\_\_  
фамилия и. о.

Курск 2026 г.

## СОДЕРЖАНИЕ

|        |                                                                                |    |
|--------|--------------------------------------------------------------------------------|----|
| 1      | Анализ предметной области                                                      | 5  |
| 1.1    | Характеристика деятельности учебно-методического центра повышения квалификации | 5  |
| 1.2    | Проблематика традиционного подхода к организации учебного процесса             | 5  |
| 1.3    | Обоснование необходимости автоматизации                                        | 7  |
| 1.4    | Анализ существующих систем автоматизации учебного процесса                     | 7  |
| 1.4.1  | Moodle                                                                         | 7  |
| 1.4.2  | Google Classroom                                                               | 8  |
| 1.4.3  | 1С:Университет                                                                 | 9  |
| 1.4.4  | Вывод по анализу аналогов                                                      | 9  |
| 1.5    | Постановка задач на разработку                                                 | 10 |
| 2      | Техническое задание                                                            | 11 |
| 2.1    | Основание для разработки                                                       | 11 |
| 2.2    | Цель и назначение разработки                                                   | 11 |
| 2.3    | Требования к функциональности                                                  | 11 |
| 2.4    | Требования к интерфейсу                                                        | 13 |
| 2.5    | Моделирование вариантов использования                                          | 14 |
| 2.5.1  | Вариант использования «Вход в систему»                                         | 16 |
| 2.5.2  | Вариант использования «Выход из системы»                                       | 17 |
| 2.5.3  | Вариант использования «Добавление нового пользователя»                         | 17 |
| 2.5.4  | Вариант использования «Редактирование пользователя»                            | 18 |
| 2.5.5  | Вариант использования «Удаление пользователя»                                  | 20 |
| 2.5.6  | Вариант использования «Просмотр списка слушателей»                             | 21 |
| 2.5.7  | Вариант использования «Добавление нового слушателя»                            | 22 |
| 2.5.8  | Вариант использования «Редактирование данных слушателя»                        | 23 |
| 2.5.9  | Вариант использования «Удаление слушателя»                                     | 24 |
| 2.5.10 | Вариант использования «Установка фильтров для отбора слушателей»               | 25 |

|        |                                                               |    |
|--------|---------------------------------------------------------------|----|
| 2.5.11 | Вариант использования «Сброс фильтров»                        | 26 |
| 2.5.12 | Вариант использования «Импорт слушателей из Excel-файла»      | 26 |
| 2.5.13 | Вариант использования «Просмотр статистики по слушателям»     | 28 |
| 2.5.14 | Вариант использования «Копирование статистики в буфер обмена» | 29 |
| 2.5.15 | Вариант использования «Просмотр журнала событий»              | 30 |
| 2.6    | Требования к надёжности                                       | 30 |
| 2.7    | Требования к информационной безопасности                      | 31 |
| 2.8    | Требования к программной и технической совместимости          | 32 |
| 2.9    | Требования к оформлению документации                          | 33 |
| 3      | Технический проект                                            | 34 |
| 3.1    | Архитектура программной системы                               | 34 |
| 3.2    | Обоснование выбора технологии проектирования                  | 35 |
| 3.2.1  | Язык программирования Python                                  | 35 |
| 3.2.2  | Микрофреймворк Flask                                          | 35 |
| 3.2.3  | СУБД PostgreSQL и библиотека pgembed                          | 36 |
| 3.2.4  | Библиотека Tkinter                                            | 37 |
| 3.2.5  | Прочие библиотеки                                             | 37 |
| 3.3    | Клиентская часть                                              | 37 |
| 3.4    | Серверная часть                                               | 39 |
| 3.4.1  | Обработка запроса                                             | 41 |
| 3.4.2  | Форматы данных                                                | 42 |
| 3.4.3  | Программный интерфейс (API)                                   | 42 |
| 3.5    | Проект базы данных программно-информационной системы          | 44 |
| 3.5.1  | Компоненты базы данных                                        | 44 |
| 3.5.2  | ER-модель данных                                              | 45 |
| 3.5.3  | Нормализация                                                  | 45 |
| 3.5.4  | Реляционная модель данных                                     | 46 |
| 3.5.5  | Описание структуры базы данных                                | 47 |
|        | СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ                              | 53 |

## ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

БД – база данных.

ДОТ – дистанционные образовательные технологии.

ИС – информационная система.

ИТ – информационные технологии.

ПО – программное обеспечение.

РП – рабочий проект.

СНИЛС – страховой номер индивидуального лицевого счёта.

СУБД – система управления базами данных.

ТЗ – техническое задание.

ТП – технический проект.

УГС – укрупнённая группа специальностей.

API (Application Programming Interface) – программный интерфейс приложения.

CSS (Cascading Style Sheets) – каскадные таблицы стилей.

ER (Entity-Relationship) – модель «сущность-связь».

HTML (HyperText Markup Language) – язык гипертекстовой разметки.

HTTP (HyperText Transfer Protocol) – протокол передачи гипертекста.

SQL (Structured Query Language) – язык структурированных запросов.

UML (Unified Modelling Language) – язык графического описания для объектного моделирования в области разработки программного обеспечения.

GUI (Graphical User Interface) — графический интерфейс пользователя.

## **1 Анализ предметной области**

### **1.1 Характеристика деятельности учебно-методического центра повышения квалификации**

Учебно-методические центры повышения квалификации и профессиональной переподготовки играют ключевую роль в обеспечении профессионального развития сотрудников различных отраслей. В условиях стремительных изменений в технологиях, рыночных реалиях необходимость в непрерывном обучении становится особенно важной. Профессионалы, чья деятельность напрямую зависит от актуальности знаний, обязаны поддерживать высокий уровень своей квалификации, чтобы оставаться конкурентоспособными и соответствовать современным требованиям.

Образовательные центры, предлагающие курсы повышения квалификации и переподготовки, становятся важнейшими инструментами для поддержания профессиональной пригодности специалистов в различных областях. Такие центры организуют обучение по дополнительным профессиональным программам, ведут учёт слушателей, преподавателей, учебных групп, а также формируют отчётность.

### **1.2 Проблематика традиционного подхода к организации учебного процесса**

В традиционном подходе к организации учебных курсов и образовательных программ существует ряд значительных проблем, которые существенно снижают эффективность работы учебно-методического центра.

**Отсутствие централизованного хранилища данных.** В настоящее время информация о слушателях, преподавателях, программах повышения квалификации и профессиональной переподготовки хранится в разрозненных электронных таблицах. Это приводит к дублированию данных, затрудняет их поиск и актуализацию. Например, при изменении статуса слушателя или выдаче документа необходимо вручную обновлять несколько файлов, что повышает риск возникновения ошибок.

**Трудоёмкость ведения отчётности.** Учебно-методический центр обязан регулярно предоставлять статистическую отчётность. Для формирования такой отчётности требуется вручную агрегировать данные из множества источников: списков слушателей, журналов учёта документов, финансовых ведомостей. Ручной подсчёт показателей (число слушателей по категориям, источникам финансирования, возрасту, полу) занимает значительное время и чреват арифметическими ошибками.

**Сложность контроля за успеваемостью и статусом слушателей.** Отсутствие единой информационной системы не позволяет оперативно отслеживать, какие программы освоены слушателем, какие документы ему выданы, является ли он педагогическим работником или руководителем. Такая информация необходима как для внутреннего учёта, так и для ответов на запросы контролирующих органов. В текущих условиях получение подобных сведений требует ручного просмотра нескольких таблиц и сопоставления данных.

**Низкая скорость обработки запросов.** При обращении сотрудника центра с запросом типа «найти всех слушателей по фамилии Иванов, обучавшихся в 2024 году» или «выдать список слушателей с определённым статусом» поиск выполняется вручную с использованием штатных средств электронных таблиц. При большом объёме данных такой подход становится крайне неэффективным и может занимать много времени.

**Отсутствие разграничения прав доступа.** Хранение данных в общих файлах на сетевом диске не позволяет гибко настраивать права доступа: администратор, оператор и простой пользователь имеют одинаковые возможности по просмотру и редактированию информации. Это создаёт риск случайного или преднамеренного изменения важных данных.

Таким образом, существующий «ручной» подход к управлению учебным процессом исчерпал свои возможности. Дальнейшее развитие центра требует внедрения специализированной информационной системы, которая автоматизирует учётные операции, ускорит формирование отчётности и повысит достоверность данных.

### **1.3 Обоснование необходимости автоматизации**

Перечисленные проблемы могут быть эффективно решены путём создания информационной системы для автоматизации процессов управления учебными курсами. Такая система должна обеспечить:

- централизованное хранение и управление данными о слушателях, преподавателях, программах и группах;
- возможность гибкой фильтрации и поиска информации по множеству критериев;
- автоматическое формирование статистических сводок;
- импорт данных из электронных таблиц (например, из файлов Excel, предоставляемых заказчиком);
- разграничение прав доступа (администраторы, операторы);
- удобный веб-интерфейс для работы с системой.

Автоматизация позволит централизовать все данные, устранить ошибки при ручном вводе, обеспечить оперативный контроль за успеваемостью слушателей и существенно сократить время на подготовку отчётности.

### **1.4 Анализ существующих систем автоматизации учебного процесса**

На рынке представлен широкий спектр информационных систем для управления обучением. Ниже рассмотрены наиболее распространённые решения, их функциональные возможности и ограничения применительно к задачам учебно-методического центра повышения квалификации.

#### **1.4.1 Moodle**

Moodle является одной из самых популярных бесплатных систем управления обучением с открытым исходным кодом. Основные функции Moodle включают:

- создание и управление курсами, лекциями, тестами и заданиями;

- форум, чат, обмен сообщениями между преподавателями и студентами;

- ведение журнала оценок и отслеживание активности пользователей;

- поддержка SCORM и других стандартов дистанционного обучения;

- разграничение ролей (администратор, преподаватель, студент).

**Избыточность для задач центра.** Moodle ориентирован на организацию полноценного дистанционного обучения с интерактивными элементами. Для учебно-методического центра, основная задача которого – учёт слушателей, программ и формирование отчётности, функционал Moodle является чрезмерным. Кроме того, Moodle не предоставляет готовых инструментов для формирования статистической отчётности, а его настройка под специфические требования потребует значительных временных и технических ресурсов.

#### 1.4.2 Google Classroom

Google Classroom – это бесплатный сервис от Google, интегрированный с Google Диском, Gmail и Google Календарём. Основные возможности:

- создание классов и заданий, прикрепление материалов из Google Диска;

- автоматическое создание папок для каждого класса и студента;

- обратная связь и оценки;

- интеграция с другими сервисами Google.

**Избыточность для задач центра.** Google Classroom не предоставляет гибких средств для ведения детального учёта слушателей. Отсутствуют механизмы формирования сложных отчётов и импорта данных из внешних систем. Сервис не предназначен для работы с персональными данными в объёме, требуемом для отчётности перед государственными органами.



### 1.4.3 1С:Университет

Программный продукт «1С:Университет» разработан для автоматизации деятельности образовательных учреждений высшего и среднего профессионального образования. Его функционал включает:

- учёт контингента студентов, успеваемости, посещаемости;
- формирование учебных планов и рабочих программ;
- расчёт нагрузки преподавателей;
- подготовка государственных отчётов (в том числе для Росстата);
- интеграция с системами «1С:Зарплата и управление персоналом».

**Избыточность для задач центра.** Система обладает высокой стоимостью внедрения и сопровождения. Она избыточна для небольшого учебно-методического центра, поскольку содержит множество модулей, не имеющих отношения к дополнительному профессиональному образованию. Кроме того, «1С:Университет» требует установки и администрирования специализированного серверного программного обеспечения, что повышает порог входа.

### 1.4.4 Вывод по анализу аналогов

Проведённый анализ существующих систем управления обучением показал, что ни одна из них в полной мере не соответствует специфике деятельности и организационно-техническим требованиям учебно-методического центра.

Универсальные образовательные платформы (Moodle, Google Classroom) обладают избыточным функционалом для организации дистанционного взаимодействия, однако не предоставляют специализированных средств для детального учёта контингента слушателей, ведения документов об образовании и формирования утверждённых форм статистической отчётности. Их адаптация под данные задачи потребовала бы значительной доработки, что сопоставимо с разработкой нового решения.

С другой стороны, комплексные системы, такие как «1С:Университет», предлагают широкие возможности для автоматизации деятельности образовательных учреждений, однако их внедрение сопряжено с высокой стоимостью программного обеспечения и необходимостью обслуживания квалифицированным персоналом. Функциональные возможности этих систем значительно превышают реальные потребности центра, что приводит к неоправданным затратам.

Таким образом, целесообразным является не внедрение готового продукта, а разработка собственной специализированной информационной системы. Это позволит создать решение, максимально адаптированное под уникальные бизнес-процессы центра, избежав при этом функциональной избыточности и высоких лицензионных расходов.

### **1.5 Постановка задач на разработку**

На основе проведённого анализа были сформулированы следующие задачи, которые должна решать разрабатываемая информационная система:

1. Обеспечить ввод, хранение и редактирование информации о слушателях, преподавателях, программах повышения квалификации и профессиональной переподготовки.
2. Реализовать механизмы фильтрации и поиска данных по различным критериям.
3. Предоставить возможность выгрузки статистических сводок.
4. Реализовать импорт исходных данных из Excel-файлов (в соответствии с предоставленными образцами).
5. Обеспечить разграничение прав доступа.
6. Создать удобный веб-интерфейс для работы с системой и десктопное приложение для управления серверной частью.

## **2 Техническое задание**

### **2.1 Основание для разработки**

Основанием для разработки является задание на выпускную квалификационную работу бакалавра «Программно-информационная система для центра повышения квалификации университета».

### **2.2 Цель и назначение разработки**

Основной целью работы является разработка и внедрение программно-информационной системы для автоматизации деятельности учебно-методического центра повышения квалификации.

Задачами разработки являются:

- создание централизованной базы данных для хранения информации о слушателях, преподавателях, программах обучения, группах и выдаваемых документах;
- реализация удобного веб-интерфейса для просмотра, добавления, редактирования и удаления данных;
- реализация гибкой системы фильтрации и поиска слушателей по множеству критериев (ФИО, СНИЛС, даты, статусы, программы, кафедры и т.д.);
- обеспечение авторизации и разграничения прав доступа (администратор, оператор);
- разработка механизма импорта данных из Excel-файлов в формате, предоставляемом заказчиком;
- автоматическое формирование статистических отчётов;
- создание десктопного приложения для управления серверной частью.

### **2.3 Требования к функциональности**

Информационная система должна обеспечивать выполнение следующих функций:

**1. Управление данными о слушателях.** Система должна позволять добавлять новых слушателей с заполнением всех необходимых полей (личные данные, обучение, финансы, документ, профессиональные характеристики, предыдущее образование), редактировать и удалять существующие записи, а также просматривать список слушателей в виде таблицы со всеми полями.

**2. Фильтрация и поиск.** Должна быть реализована возможность текстового поиска по ФИО, СНИЛС и номеру документа. Также необходима фильтрация по полу, возрасту, датам (рождения, начала обучения), наличию ДОТ, множественный выбор статусов слушателей, кафедр, программ и источников финансирования, а также фильтрация по дополнительным полям: педагогический работник, руководитель, область деятельности, квалификация.

**3. Управление пользователями.** Система должна обеспечивать авторизацию по логину и паролю с разделением прав: администратор имеет полный доступ, оператор — только просмотр. Администратор должен иметь возможность добавлять, редактировать и удалять пользователей.

**4. Импорт данных.** Должна быть реализована загрузка данных из Excel-файлов (.xlsx) со структурой, соответствующей предоставленному образцу. Перед импортом необходима валидация данных, а по окончании — вывод отчёта с указанием количества обработанных строк и ошибок.

**5. Статистика и отчётность.** Система должна формировать сводку по слушателям: общее количество, распределение по программам, статусам, источникам финансирования, полу и возрастным группам. Должна быть предусмотрена возможность копирования полученного отчёта в буфер обмена.

**6. Серверное десктоп-приложение.** Требуется реализовать приложение для запуска и остановки встроенного сервера PostgreSQL и веб-сервера Flask, отображения статуса их работы, а также просмотра логов приложения.

## 2.4 Требования к интерфейсу

Интерфейс информационной системы представляет собой веб-приложение с единым минималистичным стилем оформления, обеспечивающее интуитивно понятную навигацию. Основные элементы интерфейса включают: главную страницу со списком слушателей, панелью фильтров и кнопками управления; страницу добавления и редактирования слушателя с группировкой полей по разделам; форму авторизации; страницу управления пользователями, а также всплывающие окна для просмотра статистики и логов.

На рисунке 2.1 представлен внешний вид главной страницы веб-приложения.

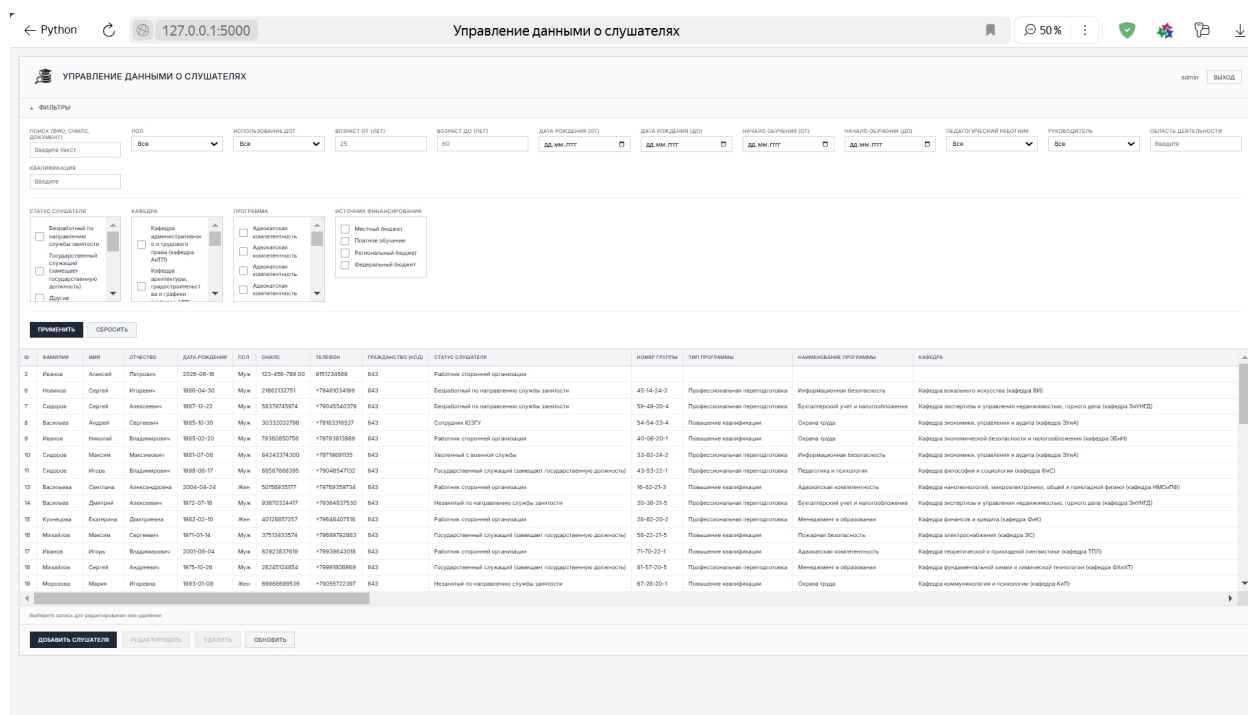


Рисунок 2.1 – Главная страница веб-приложения

Как показано на рисунке, главная страница содержит шапку с названием системы, именем текущего пользователя и кнопкой выхода, раскрывающуюся панель для фильтрации данных по различным критериям, таблицу со списком слушателей (ФИО, дата рождения, СНИЛС, статус, программа

обучения и др.), а также панель управления с кнопками «Добавить», «Редактировать», «Удалить» и «Обновить».

Вспомогательные окна просмотра журнала событий и статистики показаны на рисунке 2.2.

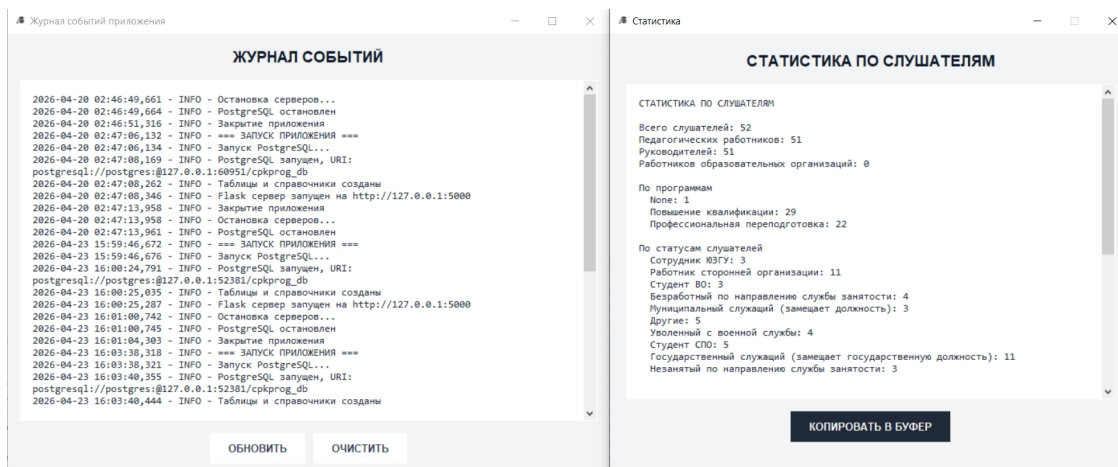


Рисунок 2.2 – Окна просмотра журнала событий и статистики

## 2.5 Моделирование вариантов использования

В системе предусмотрены две роли. Администратор обладает полными правами: просмотр, добавление, редактирование и удаление слушателей, импорт данных, формирование статистики, управление пользователями. Оператор имеет доступ только к просмотру списка слушателей без возможности внесения изменений.

Основные прецеденты (варианты использования) системы:

1. Вход в систему.
2. Просмотр списка слушателей с возможностью фильтрации и поиска.
3. Добавление нового слушателя.
4. Редактирование существующего слушателя.
5. Удаление слушателя.
6. Импорт данных из Excel-файла.
7. Просмотр статистики.
8. Управление пользователями.
9. Выход из системы.

На рисунке 2.3 представлена диаграмма вариантов использования для веб-сайта системы.

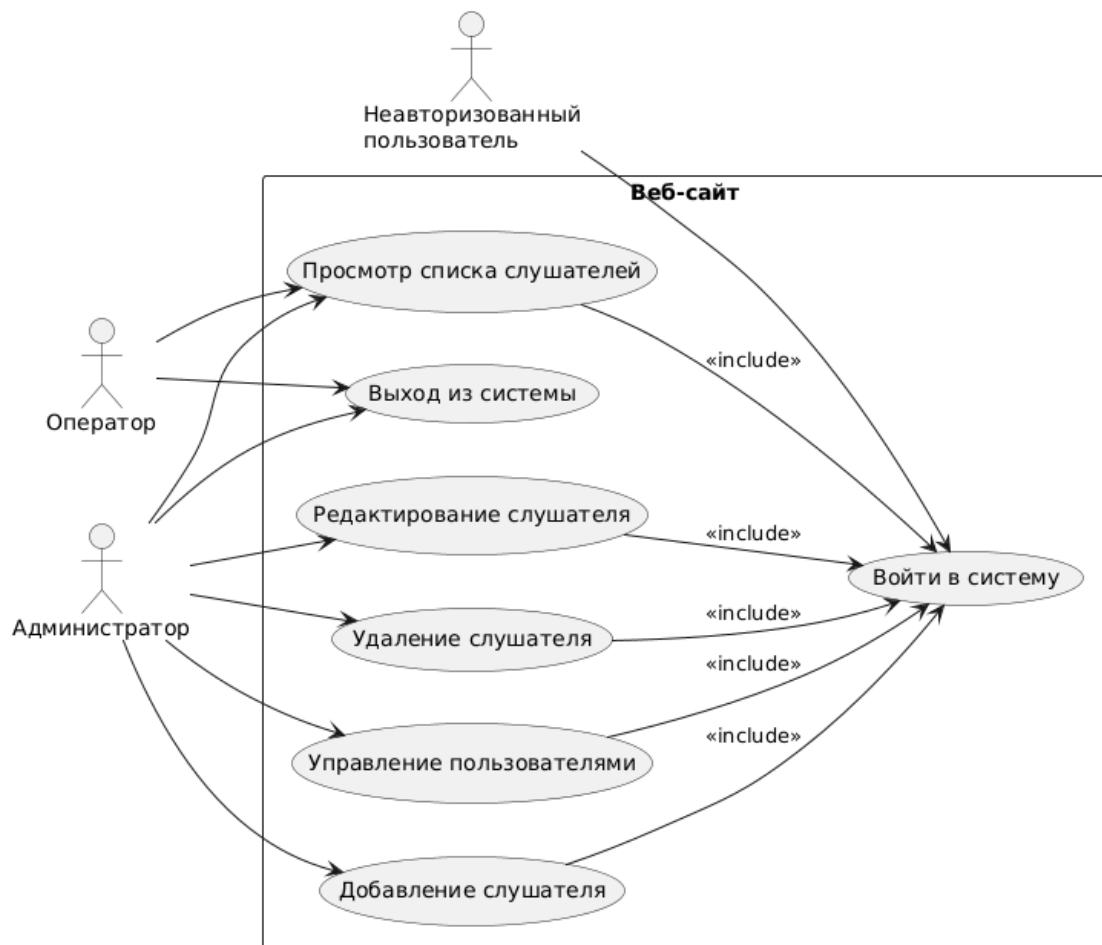


Рисунок 2.3 – Диаграмма прецедентов для веб-сайта

На рисунке 2.4 представлена диаграмма вариантов использования для десктоп-приложения, предназначенного для управления серверной частью.

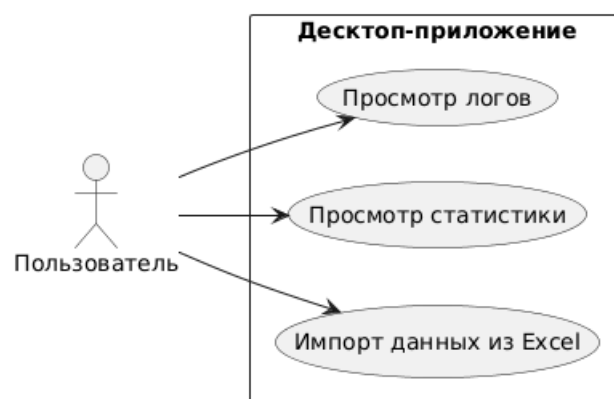


Рисунок 2.4 – Диаграмма прецедентов для десктоп-приложения

### 2.5.1 Вариант использования «Вход в систему»

**Заинтересованные лица и их требования:** пользователь (администратор или обычный пользователь) желает получить доступ к функциям системы путём аутентификации по логину и паролю.

**Предусловие:** пользователь не аутентифицирован в системе; открыта страница входа.

**Постусловие:** пользователь аутентифицирован, ему предоставлен доступ к функциям системы в соответствии с его уровнем прав (администратор или обычный пользователь).

#### **Основной успешный сценарий:**

1. Пользователь открывает страницу входа, содержащую поля «Логин» и «Пароль», а также кнопку «Войти».
2. Пользователь вводит логин и пароль в соответствующие поля.
3. Пользователь нажимает кнопку «Войти».
4. Приложение выполняет поиск учётной записи в таблице users базы данных по введённому логину.
5. Приложение извлекает хеш пароля, сохранённый для данной учётной записи.
6. Приложение выполняет сравнение введённого пароля с сохранённым хешем с использованием криптографической функции `check_password_hash`.
7. При совпадении хешей приложение создаёт сессию пользователя и перенаправляет его на главную страницу (список слушателей).

#### **Альтернативный сценарий (неверный логин или пароль):**

1. На шаге 4 основного сценария учётная запись с введённым логином не найдена, либо на шаге 6 хеши не совпадают.
2. Приложение отображает сообщение об ошибке: «Неверный логин или пароль».
3. Пользователь остаётся на странице входа и может повторить попытку.



### 2.5.2 Вариант использования «Выход из системы»

**Заинтересованные лица и их требования:** аутентифицированный пользователь желает завершить сеанс работы с системой.

**Предусловие:** пользователь аутентифицирован в системе.

**Постусловие:** сессия пользователя уничтожена, доступ к защищённым страницам системы заблокирован до повторной аутентификации.

**Основной успешный сценарий:**

1. Пользователь нажимает кнопку «ВЫХОД», расположенную в верхней панели интерфейса.
2. Приложение вызывает функцию `logout_user`, которая уничтожает текущую сессию пользователя.
3. Приложение перенаправляет пользователя на страницу входа.

### 2.5.3 Вариант использования «Добавление нового пользователя»

**Заинтересованные лица и их требования:** администратор системы желает создать новую учётную запись для предоставления доступа к системе другому пользователю.

**Предусловие:** администратор аутентифицирован в системе; открыта страница управления пользователями (`/admin/users`).

**Постусловие:** новая учётная запись создана в базе данных; пользователь может войти в систему с указанными логином и паролем.

**Основной успешный сценарий:**

1. Администратор нажимает кнопку «ДОБАВИТЬ ПОЛЬЗОВАТЕЛЯ» на странице управления пользователями.
2. Приложение отображает форму с полями «Логин», «Пароль» и флажком «Администратор».
3. Администратор вводит логин и пароль для нового пользователя, при необходимости устанавливает флажок «Администратор».
4. Администратор нажимает кнопку «СОХРАНИТЬ».
5. Приложение проверяет, что поля «Логин» и «Пароль» не пусты.

6. Приложение выполняет запрос к базе данных для проверки уникальности логина.

7. При отсутствии совпадений приложение вычисляет хеш пароля с использованием функции `generate_password_hash`.

8. Приложение вставляет новую запись в таблицу `users` с указанными логином, хешем пароля и признаком администратора.

9. Приложение отображает сообщение об успешном добавлении пользователя и перенаправляет администратора на страницу списка пользователей.

#### **Альтернативный сценарий (логин уже существует):**

1. На шаге 6 основного сценария найден пользователь с таким же логином.

2. Приложение отображает сообщение об ошибке: «Пользователь с таким логином уже существует».

3. Администратор остаётся на странице формы и может изменить логин.

#### **Альтернативный сценарий (пустые обязательные поля):**

1. На шаге 5 основного сценария обнаружено, что поле «Логин» или «Пароль» не заполнено.

2. Приложение отображает сообщение об ошибке: «Логин и пароль обязательны».

3. Администратор остаётся на странице формы и может заполнить недостающие поля.

### **2.5.4 Вариант использования «Редактирование пользователя»**

**Заинтересованные лица и их требования:** администратор системы желает изменить данные существующей учётной записи (логин, пароль или наличие административных прав).

**Предусловие:** администратор аутентифицирован в системе; открыта страница управления пользователями (`/admin/users`); в таблице отображается хотя бы одна учётная запись.

**Постусловие:** данные учётной записи обновлены в таблице users; при изменении логина или пароля пользователь должен использовать новые учётные данные для входа.

**Основной успешный сценарий:**

1. Администратор нажимает кнопку «РЕДАКТИРОВАТЬ» в строке соответствующего пользователя. Кнопка содержит ссылку вида /admin/users/edit/<id>, где <id> — идентификатор пользователя, полученный из базы данных при формировании таблицы.
2. Приложение выполняет запрос к базе данных для получения текущих данных пользователя по идентификатору <id>.
3. Приложение отображает форму редактирования, содержащую поля «Логин» (заполнено текущим значением), «Пароль» (пустое, с подсказкой «оставьте пустым, чтобы не менять»), и флажок «Администратор» (установлен в соответствии с текущим значением).
4. Администратор изменяет требуемые поля: вводит новый логин, и/или новый пароль, и/или изменяет состояние флажка «Администратор».
5. Администратор нажимает кнопку «СОХРАНИТЬ».
6. Приложение проверяет, что поле «Логин» не пусто.
7. Приложение выполняет запрос к базе данных для проверки уникальности нового логина (исключая текущего пользователя с идентификатором <id>).
8. Если введён новый пароль, приложение вычисляет его хеш с использованием функции generate\_password\_hash; если поле пароля оставлено пустым, существующий хеш сохраняется без изменений.
9. Приложение выполняет обновление записи в таблице users.
10. Приложение отображает сообщение «Пользователь обновлён» и перенаправляет администратора на страницу списка пользователей.

**Альтернативный сценарий (логин занят другим пользователем):**

1. На шаге 7 основного сценария обнаружено, что введённый логин уже принадлежит другому пользователю.

2. Приложение отображает сообщение об ошибке: «Пользователь с таким логином уже существует».

3. Администратор остаётся на странице формы и может изменить логин на другой. Введённые данные, кроме логина, сохраняются в форме.

**Альтернативный сценарий (пользователь не найден):**

1. На шаге 2 основного сценария запрос к базе данных не возвращает результат (запись была удалена другим администратором между загрузкой страницы и нажатием кнопки, либо идентификатор был изменён вручную в адресной строке).

2. Приложение возвращает HTTP-ответ с кодом 404 (Not Found) и отображает сообщение «Пользователь не найден».

**2.5.5 Вариант использования «Удаление пользователя»**

**Заинтересованные лица и их требования:** администратор системы желает удалить учётную запись пользователя, которому больше не требуется доступ к системе.

**Предусловие:** администратор аутентифицирован в системе; открыта страница управления пользователями (/admin/users); существует удаляемая учётная запись.

**Постусловие:** учётная запись удалена из таблицы users; пользователь больше не может войти в систему.

**Основной успешный сценарий:**

1. Администратор нажимает кнопку «УДАЛИТЬ» в строке соответствующего пользователя.

2. Приложение отображает диалоговое окно подтверждения с сообщением «Удалить пользователя?».

3. Администратор подтверждает удаление нажатием кнопки «ОК».

4. Приложение проверяет, что идентификатор удаляемого пользователя не совпадает с идентификатором текущего администратора.

5. Приложение выполняет запрос `DELETE FROM users WHERE id = %s` к базе данных.

6. Приложение отображает сообщение «Пользователь удалён» и обновляет список пользователей.

**Альтернативный сценарий (попытка удаления самого себя):**

1. На шаге 4 основного сценария обнаружено, что администратор пытается удалить собственную учётную запись.

2. Приложение отображает сообщение об ошибке: «Нельзя удалить самого себя».

3. Учётная запись не удаляется, администратор остаётся на странице списка пользователей.

### **2.5.6 Вариант использования «Просмотр списка слушателей»**

**Заинтересованные лица и их требования:** пользователь желает ознакомиться с перечнем слушателей, содержащихся в базе данных системы.

**Предусловие:** пользователь аутентифицирован в системе.

**Постусловие:** на экране отображена таблица со списком слушателей.

**Основной успешный сценарий:**

1. Пользователь открывает главную страницу приложения.

2. Приложение запрашивает из базы данных список всех слушателей, включая связанные данные (группа, программа, кафедра, документ, финансирование и т. д.).

3. Приложение запрашивает из базы данных значения справочников (статусы, кафедры, программы, источники финансирования) для заполнения панели фильтров.

4. Приложение формирует HTML-страницу, содержащую таблицу со слушателями и панель фильтров.

5. Браузер отображает страницу. Таблица имеет фиксированную высоту с вертикальной прокруткой, заголовки колонок зафиксированы и остаются видимыми при прокрутке.

### **2.5.7 Вариант использования «Добавление нового слушателя»**

**Заинтересованные лица и их требования:** пользователь желает внести в базу данных информацию о новом слушателе.

**Предусловие:** пользователь аутентифицирован в системе.

**Постусловие:** новый слушатель и все связанные с ним данные сохранены в базе данных; слушатель отображается в общем списке.

**Основной успешный сценарий:**

1. Пользователь нажимает кнопку «ДОБАВИТЬ СЛУШАТЕЛЯ» на главной странице.
2. Приложение отображает форму добавления слушателя, содержащую пять тематических разделов: «Личные данные», «Обучение», «Финансы», «Документ о квалификации», «Профессиональные характеристики», «Предыдущее образование». Форма включает текстовые поля, выпадающие списки, поля выбора дат и числовые поля.
3. Пользователь заполняет поля формы.
4. Пользователь нажимает кнопку «СОХРАНИТЬ».
5. Приложение выполняет валидацию введённых данных.
6. При успешной валидации приложение сохраняет данные слушателя и всех связанных записей в базе данных.
7. Приложение перенаправляет пользователя на главную страницу, где отображается обновлённый список слушателей.

**Альтернативный сценарий (ошибка валидации):**

1. На шаге 5 основного сценария валидация обнаруживает ошибки.
2. Приложение отображает форму с сообщением об ошибке и сохраняет все ранее введённые пользователем данные в полях формы, чтобы пользователю не требовалось заполнять их заново.

## **2.5.8 Вариант использования «Редактирование данных слушателя»**

**Заинтересованные лица и их требования:** пользователь желает изменить данные ранее добавленного слушателя.

**Предусловие:** пользователь аутентифицирован в системе; слушатель с указанным идентификатором существует в базе данных.

**Постусловие:** данные слушателя и всех связанных записей обновлены в базе данных.

### **Основной успешный сценарий:**

1. Пользователь на главной странице выбирает слушателя, щёлкнув по соответствующей строке таблицы. Выбранная строка подсвечивается.
2. Кнопка «РЕДАКТИРОВАТЬ», ранее неактивная, становится активной.
3. Пользователь нажимает кнопку «РЕДАКТИРОВАТЬ».
4. Приложение загружает из базы данных текущие данные выбранного слушателя.
5. Приложение отображает форму редактирования, идентичную форме добавления, с полями, предзаполненными текущими значениями.
6. Пользователь изменяет требуемые поля.
7. Пользователь нажимает кнопку «СОХРАНИТЬ ИЗМЕНЕНИЯ».
8. Приложение выполняет валидацию изменённых данных.
9. При успешной валидации приложение обновляет данные слушателя и всех связанных записей в базе данных.
10. Приложение перенаправляет пользователя на главную страницу с обновлённым списком.

### **Альтернативный сценарий (слушатель не найден):**

1. На шаге 4 основного сценария запись с указанным идентификатором не обнаружена.
2. Приложение отображает сообщение «Студент не найден».

**Альтернативный сценарий (ошибка валидации при редактировании):**

1. На шаге 8 основного сценария валидация обнаруживает ошибки.
2. Приложение отображает форму редактирования с сообщением об ошибке; некорректные значения сохраняются в полях формы, остальные поля содержат актуальные данные из базы данных.

### **2.5.9 Вариант использования «Удаление слушателя»**

**Заинтересованные лица и их требования:** пользователь желает удалить запись о слушателе из базы данных.

**Предусловие:** пользователь аутентифицирован в системе; на главной странице отображается список слушателей; выбран конкретный слушатель.

**Постусловие:** запись о слушателе и все связанные с ней данные удалены из базы данных; слушатель больше не отображается в списке.

**Основной успешный сценарий:**

1. Пользователь на главной странице выбирает слушателя, щёлкнув по соответствующей строке таблицы. Выбранная строка подсвечивается.
2. Кнопка «УДАЛИТЬ», ранее неактивная, становится активной.
3. Пользователь нажимает кнопку «УДАЛИТЬ».
4. Приложение отображает диалоговое окно подтверждения с сообщением «Удалить слушателя?».
5. Пользователь подтверждает удаление.
6. Приложение выполняет удаление записи слушателя из базы данных.
7. Приложение перенаправляет пользователя на главную страницу с обновлённым списком слушателей.

**Альтернативный сценарий (отмена удаления):**

1. На шаге 4 основного сценария пользователь отказывается от удаления.
2. Никаких изменений в базе данных не производится, страница остаётся в текущем состоянии.



### **2.5.10 Вариант использования «Установка фильтров для отбора слушателей»**

**Заинтересованные лица и их требования:** пользователь желает сократить объём отображаемых данных, отобрав слушателей по одному или нескольким критериям.

**Предусловие:** пользователь аутентифицирован в системе; открыта главная страница со списком слушателей.

**Постусловие:** таблица слушателей обновлена и содержит только записи, удовлетворяющие заданным критериям. Параметры фильтрации сохранены в URL страницы.

#### **Основной успешный сценарий:**

1. Пользователь раскрывает панель фильтров, нажав на заголовок «ФИЛЬТРЫ». При первом открытии страницы панель уже раскрыта.
2. Пользователь устанавливает один или несколько критериев фильтрации. Панель фильтров предоставляет следующие типы элементов ввода:
  - текстовое поле для поиска по подстроке (поиск по ФИО, СНИЛС, номеру документа);
  - выпадающие списки для выбора одного значения из predetermined набора (пол, использование ДОТ, педагогический работник, руководитель);
  - числовые поля и поля выбора дат для задания диапазонов (возраст, дата рождения, дата начала обучения);
  - текстовые поля для поиска по справочным значениям (область деятельности, квалификация);
  - списки с флажками для множественного выбора (статусы слушателей, кафедры, программы, источники финансирования).
3. Пользователь нажимает кнопку «ПРИМЕНИТЬ».
4. Приложение формирует запрос к базе данных с учётом всех заданных критериев. Различные критерии объединяются логической операци-

ей «И», при множественном выборе внутри одного критерия — операцией «ИЛИ».

5. Приложение получает отфильтрованный список слушателей и обновляет содержимое таблицы.

6. Параметры фильтрации сохраняются в URL страницы, что позволяет обновить страницу без потери настроек.

#### **2.5.11 Вариант использования «Сброс фильтров»**

**Заинтересованные лица и их требования:** пользователь желает отменить все установленные ранее критерии фильтрации и вернуться к просмотру полного списка слушателей.

**Предусловие:** пользователь аутентифицирован в системе; открыта главная страница; применён хотя бы один критерий фильтрации.

**Постусловие:** все фильтры сброшены; таблица отображает полный список слушателей.

##### **Основной успешный сценарий:**

1. Пользователь нажимает кнопку «СБРОСИТЬ», расположенную рядом с кнопкой «ПРИМЕНИТЬ» на панели фильтров.

2. Приложение выполняет переход на главную страницу без параметров фильтрации в URL (маршрут /).

3. Приложение загружает полный список слушателей из базы данных.

4. Таблица обновляется, отображая всех слушателей. Все элементы панели фильтров возвращаются в исходное состояние (пустые поля ввода, выпадающие списки в значении «Все», все флажки сняты).

#### **2.5.12 Вариант использования «Импорт слушателей из Excel-файла»**

**Заинтересованные лица и их требования:** пользователь желает загрузить в базу данных множество записей о слушателях из заранее подготовленного файла в формате Excel (XLSX).

**Предусловие:** пользователь находится в графическом интерфейсе программы; база данных запущена.

**Постусловие:** корректные записи из файла добавлены в базу данных; ошибочные записи пропущены; информация о результатах импорта отображена пользователю; ошибки записаны в лог-файл.

**Основной успешный сценарий:**

1. Пользователь нажимает кнопку «ЗАГРУЗИТЬ ИЗ EXCEL» в главном окне программы.
2. Приложение открывает стандартный диалог выбора файла с фильтром по расширению .xlsx.
3. Пользователь выбирает файл и подтверждает выбор.
4. Приложение загружает справочники из базы данных для последующего сопоставления текстовых значений с идентификаторами.
5. Приложение читает содержимое Excel-файла. Все ячейки интерпретируются как строки.
6. Приложение выполняет сопоставление заголовков столбцов файла с внутренними ключами системы (например, заголовок «Фамилия получателя» сопоставляется с ключом last\_name).
7. Для каждой строки файла приложение формирует словарь данных, аналогичный данным формы веб-интерфейса, и преобразует текстовые значения справочных полей в соответствующие идентификаторы.
8. Для каждой строки выполняется валидация данных по тем же правилам, что и при добавлении через веб-интерфейс.
9. При успешной валидации данные вставляются в базу данных.
10. При обнаружении ошибок валидации или ошибок базы данных строка пропускается, информация об ошибке записывается в лог-файл import\_errors.log.
11. По завершении обработки всех строк приложение отображает окно с результатами импорта: общее количество строк в файле, количество успешно добавленных записей, количество ошибок, текст последней ошибки.

**Альтернативный сценарий (файл не выбран):**

1. На шаге 2 основного сценария пользователь закрывает диалог выбора файла, не выбрав файл.

2. Импорт не производится, приложение остаётся в исходном состоянии.

**Альтернативный сценарий (ошибка чтения файла):**

1. На шаге 5 основного сценария возникает ошибка при чтении файла (файл повреждён, не является Excel-файлом или имеет неверную структуру).

2. Приложение отображает сообщение об ошибке. Импорт прекращается.

**2.5.13 Вариант использования «Просмотр статистики по слушателям»**

**Заинтересованные лица и их требования:** пользователь желает получить сводную аналитическую информацию о слушателях, содержащихся в базе данных.

**Предусловие:** база данных запущена; пользователь находится в графическом интерфейсе программы.

**Постусловие:** на экране отображено окно со статистическим отчётом.

**Основной успешный сценарий:**

1. Пользователь нажимает кнопку «СТАТИСТИКА» в главном окне программы.

2. Приложение выполняет серию запросов к базе данных для сбора статистических показателей:

- общее количество слушателей;
- количество педагогических работников;
- количество руководителей;
- количество работников образовательных организаций;
- распределение слушателей по типам программ;
- распределение по статусам слушателей;
- распределение по источникам финансирования;
- распределение по полу;

- распределение по возрастным группам;
- количество студентов высшего и среднего профессионального образования.

3. Приложение формирует текстовый отчёт, содержащий все собранные показатели.

4. Приложение отображает отчёт в отдельном окне. Текст отчёта доступен для просмотра с вертикальной прокруткой.

**Альтернативный сценарий (база данных не запущена):**

1. На шаге 1 основного сценария кнопка «СТАТИСТИКА» недоступна, так как её состояние зависит от статуса сервера.
2. Пользователь не может инициировать просмотр статистики до запуска базы данных.

**Альтернативный сценарий (ошибка получения статистики):**

1. На шаге 2 основного сценария возникает ошибка при выполнении запросов к базе данных.
2. Приложение отображает окно с сообщением об ошибке.

#### **2.5.14 Вариант использования «Копирование статистики в буфер обмена»**

**Заинтересованные лица и их требования:** пользователь желает скопировать текст статистического отчёта для последующей вставки в другой документ.

**Предусловие:** открыто окно со статистическим отчётом.

**Постусловие:** текст отчёта помещён в буфер обмена операционной системы.

**Основной успешный сценарий:**

1. Пользователь нажимает кнопку «КОПИРОВАТЬ В БУФЕР» в окне статистики.
2. Приложение копирует полный текст статистического отчёта в буфер обмена.

3. Приложение отображает кратковременное подтверждение «Скопировано!» рядом с кнопкой.

### **2.5.15 Вариант использования «Просмотр журнала событий»**

**Заинтересованные лица и их требования:** пользователь (администратор системы) желает ознакомиться с журналом событий приложения для диагностики ошибок или отслеживания истории запусков и остановок.

**Предусловие:** программа запущена.

**Постусловие:** на экране отображено отдельное окно с содержимым файла журнала событий.

**Основной успешный сценарий:**

1. Пользователь нажимает кнопку «ЖУРНАЛ СОБЫТИЙ» в главном окне программы.
2. Приложение открывает отдельное окно, содержащее текстовое поле с прокруткой.
3. Приложение читает содержимое файла app.log и отображает его в текстовом поле.
4. Каждая запись журнала содержит временную метку, уровень важности и текст сообщения.

**Дополнительные возможности:**

**Обновление журнала.** Пользователь может нажать кнопку «ОБНОВИТЬ» в окне журнала, чтобы загрузить актуальное содержимое файла. Это позволяет отслеживать события в реальном времени, не закрывая окно.

**Очистка журнала.** Пользователь может нажать кнопку «ОЧИСТИТЬ» для удаления всех записей из файла журнала. После очистки содержимое окна обновляется и становится пустым.

## **2.6 Требования к надёжности**

Система должна обеспечивать надёжное хранение и обработку данных при различных режимах работы.

**Целостность данных.** База данных PostgreSQL поддерживает механизм транзакций, гарантирующий, что операции добавления, редактирования и удаления данных либо выполняются полностью, либо откатываются в исходное состояние при возникновении ошибки. Это исключает появление частично сохранённых или противоречивых данных.

**Устойчивость к сбоям.** При аварийном завершении работы сервера (например, при отключении электропитания) PostgreSQL автоматически выполняет восстановление базы данных при следующем запуске. Все успешно сохранённые транзакции восстанавливаются, незавершённые — откатываются.

**Контроль целостности на уровне приложения.** На уровне веб-приложения реализована валидация вводимых данных: проверка форматов дат, СНИЛС, телефона, кода гражданства, стоимости обучения и других полей. Это предотвращает попадание некорректных данных в базу.

**Логирование ошибок.** Все критические ошибки и исключения, возникающие при работе системы, записываются в файл журнала (app.log). Лог содержит временную метку, тип ошибки и описание причины, что облегчает диагностику неполадок.

**Резервное копирование.** Данные системы хранятся в файлах базы данных PostgreSQL (папка pgdata). Для обеспечения сохранности информации администратор может выполнить резервное копирование простым копированием данной папки или через штатные средства PostgreSQL (pg\_dump). Восстановление данных осуществляется обратным копированием.

## 2.7 Требования к информационной безопасности

Система должна обеспечивать защиту данных от несанкционированного доступа и разграничение прав пользователей.

**Авторизация.** Доступ к веб-приложению осуществляется только после ввода логина и пароля. Форма авторизации является единственной точкой входа для неавторизованных пользователей.

**Хранение паролей.** Пароли пользователей не хранятся в базе данных в открытом виде. Для их хранения используется хеширование. При авторизации введенный пароль сверяется с хешем, хранящимся в базе.

**Разграничение прав доступа.** В системе предусмотрены две роли. Администратор обладает полными правами, в то время как оператор имеет доступ только к просмотру списка слушателей без возможности внесения изменений. После успешной авторизации пользователь получает права, соответствующие его роли. Проверка прав выполняется на серверной стороне перед выполнением любого действия, что исключает возможность обхода ограничений через клиентский код.

**Защита от несанкционированного доступа.** Десктоп-приложение не требует авторизации, так как предполагается, что доступ к нему имеют только уполномоченные сотрудники. Серверная часть принимает подключения только на локальном интерфейсе (127.0.0.1), что ограничивает доступ к системе из внешней сети.

**Защита от SQL-инъекций.** Все запросы к базе данных выполняются с использованием параметризованных SQL-запросов (библиотека `psycopg2`), что исключает возможность внедрения произвольного SQL-кода через пользовательский ввод.

## 2.8 Требования к программной и технической совместимости

Система должна функционировать в типовой офисной среде и не предъявлять завышенных требований к оборудованию.

**Операционная система.** Собранное приложение является автономным и работает под управлением операционной системы Windows. Возможна работа под управлением Linux или macOS при условии сборки приложения на соответствующей платформе (исполняемый файл, собранный в среде Windows, не может быть запущен в других операционных системах). Сборка выполняется с использованием PyInstaller и включает все необходимые компоненты: интерпретатор Python, библиотеки и бинарные файлы PostgreSQL.

**Аппаратные требования:**



- процессор: 2 ГГц и выше;
- оперативная память: 4 ГБ и более;
- свободное дисковое пространство: 500 МБ для программы, дополнительно — для хранения базы данных.

**Программное обеспечение.** Для работы веб-интерфейса необходим любой современный браузер: Google Chrome, Mozilla Firefox, Microsoft Edge. Десктоп-приложение не требует установки дополнительного программного обеспечения — PostgreSQL и Flask встраиваются непосредственно в приложение с использованием библиотеки pgembed, а интерпретатор Python не требуется, так как он упакован в исполняемый файл.

**Сетевые требования.** Веб-сервер слушает порт 5000 на всех сетевых интерфейсах (0.0.0.0), что позволяет подключаться к системе не только с локального компьютера, но и с других устройств в локальной сети.

**Требования к сопровождающей документации.** Система поставляется в виде исполняемого файла и не требует установки. Пользователю достаточно запустить файл, нажать кнопку «Запустить сервер» и дождаться открытия браузера. Для администратора предусмотрен файл журнала событий и возможность просмотра логов через интерфейс десктоп-приложения.

## 2.9 Требования к оформлению документации

Разработка программной документации и программного изделия должна производиться согласно ГОСТ 19.102-77 и ГОСТ 34.601-90. Единая система программной документации.

### 3 Технический проект

#### 3.1 Архитектура программной системы

Приложение построено по архитектуре «клиент-сервер». Серверная часть включает встроенную СУБД PostgreSQL, развёртываемую через библиотеку pgembed, и веб-сервер на основе микрофреймворка Flask. Клиентская часть представлена десктопным приложением (Tkinter) и веб-браузером, через который осуществляется основная работа с системой.

Локальное использование обосновано спецификой задачи. Для центра повышения квалификации удобно использовать локальный веб-сервер: несколько сотрудников одновременно работают с общей базой данных через браузер. Администратор, запускающий приложение, управляет сервером кнопками запуска и остановки. Остальные пользователи подключаются по локальному IP-адресу компьютера администратора на порт 5000.

Общая схема взаимодействия компонентов приложения показана на рисунке 3.1.

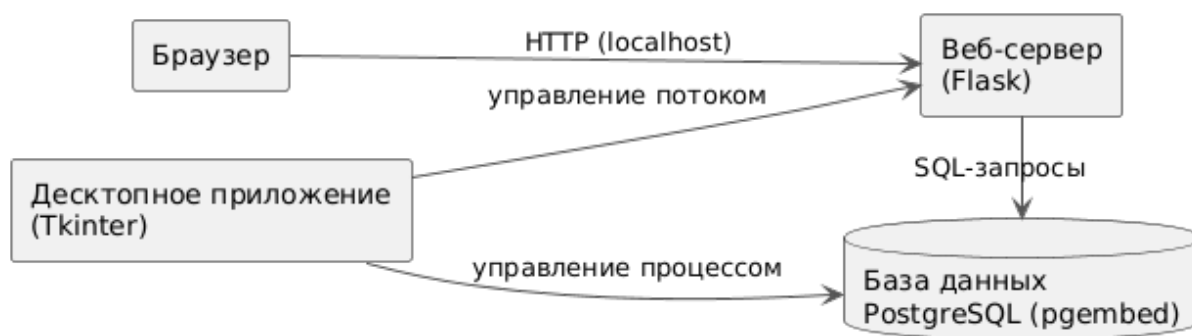


Рисунок 3.1 – Общая схема взаимодействия компонентов

Десктопное приложение (Tkinter) отвечает за управление жизненным циклом серверной части. При запуске оно инициализирует PostgreSQL, создаёт базу данных (если она отсутствует), развёртывает структуру таблиц с помощью модуля schema.py и заполняет справочники начальными значениями. Затем запускается веб-сервер Flask в отдельном потоке и открывает-

ся браузер с интерфейсом приложения. При завершении работы десктопное приложение корректно останавливает веб-сервер и процесс PostgreSQL.

Серверная часть функционирует локально. Встроенная СУБД PostgreSQL использует предопределённые параметры подключения: имя пользователя — postgres, пароль — пустой, имя базы данных — crkprog\_db. Данные физически хранятся в подкаталоге ./pgdata каталога приложения.

### **3.2 Обоснование выбора технологии проектирования**

При выборе технологического стека учитывались следующие критерии: возможность автономной работы без установки дополнительного ПО, простота развёртывания, наличие необходимых библиотек и средств для работы с базой данных и веб-интерфейсом.

#### **3.2.1 Язык программирования Python**

Основным языком разработки выбран Python 3.13. Данный язык предоставляет все необходимые для проекта библиотеки: psycopg2 для взаимодействия с PostgreSQL, Flask для создания веб-интерфейса, Tkinter для графического интерфейса пользователя. Ключевым преимуществом стало наличие библиотеки pgembed, которая позволяет встроить PostgreSQL непосредственно в приложение, избавляя пользователя от необходимости устанавливать и настраивать СУБД вручную. Дополнительным плюсом является кроссплатформенность Python, что при минимальных доработках позволит запускать приложение на различных операционных системах.

#### **3.2.2 Микрофреймворк Flask**

В качестве веб-фреймворка выбран Flask — легковесный фреймворк на языке Python, позволяющий запустить веб-сервер с минимальными усилиями. Flask не требует установки внешнего веб-сервера и обслуживает HTTP-запросы самостоятельно.

Встроенный шаблонизатор Jinja2 отделяет HTML-верстку от программной логики. В шаблонах размещаются метки-заполнители, на место ко-

торых Flask подставляет данные из базы данных — списки слушателей, названия справочников, сообщения об ошибках. Это избавляет от необходимости формировать HTML-код внутри Python-сценариев и упрощает поддержку интерфейса.

Механизм маршрутизации связывает URL-адреса с функциями-обработчиками. Например, запрос к `/add_student` направляется к функции добавления слушателя, а запрос к `/edit_student/5` — к функции редактирования слушателя с идентификатором 5. Параметры URL извлекаются автоматически.

Для аутентификации используется расширение Flask-Login с кастомным декоратором `@admin_required`, проверяющим наличие у пользователя прав администратора.

Библиотека `werkzeug`, на которой построен Flask, предоставляет класс `make_server`, позволяющий запустить веб-сервер в отдельном потоке внутри графического интерфейса пользователя. Это даёт возможность управлять запуском и остановкой сервера через графический интерфейс.

### 3.2.3 СУБД PostgreSQL и библиотека `pgembed`

В качестве системы управления базами данных выбрана PostgreSQL. Эта СУБД обеспечивает полную поддержку внешних ключей с каскадными операциями (`ON DELETE CASCADE`), благодаря чему при удалении слушателя автоматически удаляются все связанные записи — документы, финансирование, характеристики, предыдущее образование. Поддержка транзакций с возможностью отката гарантирует, что при сбое во время операции добавления или редактирования данные не будут частично сохранены.

Ключевым компонентом проекта является библиотека `pgembed`. Она встраивает PostgreSQL непосредственно в десктопное приложение: при первом запуске автоматически развёртывает экземпляр СУБД, создаёт базу данных и инициализирует структуру таблиц. Пользователю не требуется устанавливать PostgreSQL отдельно или настраивать параметры подключения.

Для эффективной работы с базой данных используется пул соединений. При запуске создаётся от 2 до 10 потокобезопасных соединений, которые переиспользуются между запросами. Это исключает накладные расходы на постоянное открытие и закрытие подключений и позволяет обслуживать несколько одновременных запросов от разных вкладок браузера.

### **3.2.4 Библиотека Tkinter**

Графический интерфейс пользователя реализован на библиотеке Tkinter. Она входит в стандартную поставку Python, поэтому не требует установки дополнительных зависимостей и не увеличивает размер дистрибутива. Функциональности Tkinter достаточно для создания интерфейса с кнопками управления сервером, отображения статуса, а также дополнительных окон для просмотра статистики, журнала событий и выбора файла импорта.

### **3.2.5 Прочие библиотеки**

Для чтения и обработки Excel-файлов при импорте данных используется библиотека pandas. Хеширование паролей пользователей выполняется средствами werkzeug (generate\_password\_hash, check\_password\_hash). Логирование событий приложения реализовано стандартным модулем Python logging с записью в файл app.log и дублированием в консоль.

## **3.3 Клиентская часть**

Клиентская часть включает десктопное приложение на Tkinter и браузеры пользователей. Десктопное приложение предоставляет графический интерфейс для управления сервером и дополнительные функции, не связанные напрямую с веб-интерфейсом. На рисунке 3.2 приведена диаграмма компонентов клиентской части.

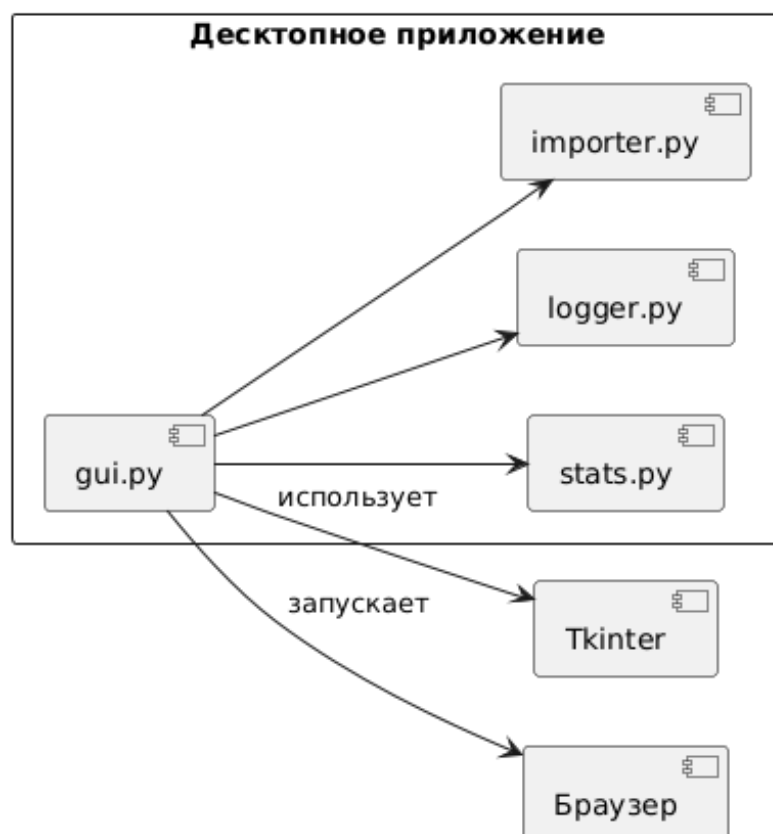


Рисунок 3.2 – Диаграмма компонентов клиентской части

Внешней платформой для десктопного приложения является библиотека Tkinter, обеспечивающая создание окон, кнопок и других элементов интерфейса. Основным компонентом является `gui.py` — главное окно приложения. Оно содержит кнопки «Запустить сервер» и «Остановить сервер», статусную строку, а также кнопки вызова дополнительных функций: «Статистика», «Журнал событий», «Загрузить из Excel», «Пользователи».

При нажатии на кнопку запуска `gui.py` выполняет последовательную инициализацию: запускает процесс PostgreSQL через `connection.py`, создаёт базу данных и таблицы через `schema.py`, запускает веб-сервер Flask в отдельном потоке и открывает браузер. При остановке последовательно завершает работу Flask, закрывает пул соединений и останавливает PostgreSQL.

Модуль `stats.py` вызывается при нажатии кнопки «Статистика». Он выполняет несколько SQL-запросов к базе данных и формирует текстовый отчёт, который отображается в отдельном окне Tkinter с возможностью копирования в буфер обмена.

Модуль `logger.py` обеспечивает просмотр и очистку журнала событий. Кнопка «Журнал событий» открывает окно, в которое загружается содержимое файла `app.log`. Пользователь может обновить содержимое или полностью очистить файл.

Модуль `importer.py` реализует импорт данных из Excel-файлов. По нажатию кнопки «Загрузить из Excel» открывается стандартный диалог выбора файла, после чего модуль считывает данные, выполняет сопоставление столбцов, валидацию и вставку записей в базу данных.

Все перечисленные модули клиентской части не зависят от веб-сервера Flask и взаимодействуют с базой данных напрямую через `connection.py`.

### **3.4 Серверная часть**

Серверная часть реализует веб-интерфейс системы и содержит всю бизнес-логику обработки данных. На рисунке 3.3 представлена диаграмма компонентов серверной части.

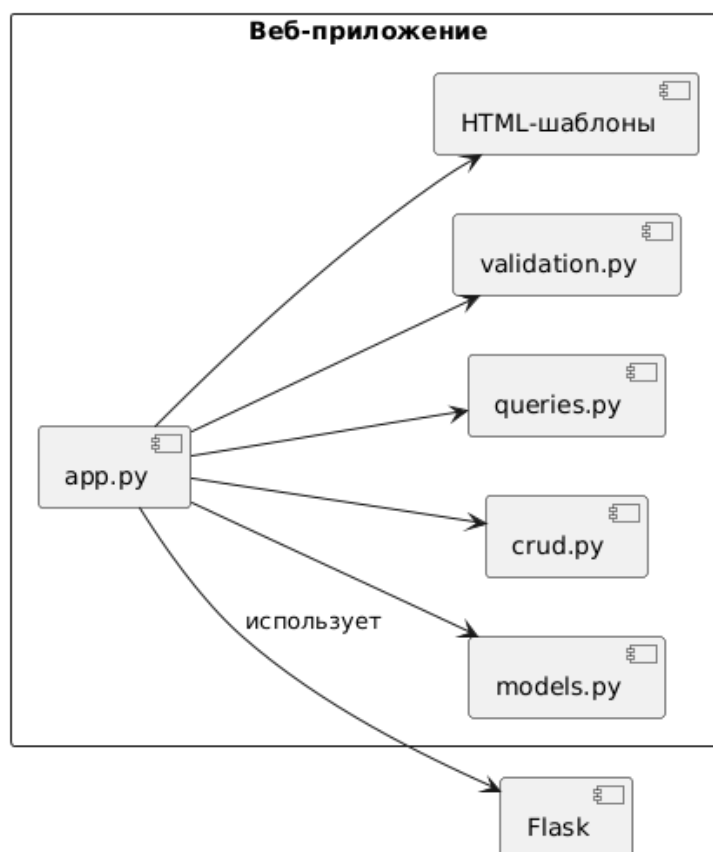


Рисунок 3.3 – Диаграмма компонентов серверной части

Внешней средой для серверной части является веб-фреймворк Flask, предоставляющий маршрутизацию, обработку запросов и шаблонизатор Jinja2. Центральный компонент `app.py` определяет все маршруты: для работы со слушателями (`/`, `/add_student`, `/edit_student/<id>`, `/delete_student/<id>`), аутентификации (`/login`, `/logout`) и управления пользователями (`/admin/users`).

Аутентификация реализована с помощью расширения Flask-Login. При входе вызывается `login_user`, создающая защищённую сессию. При каждом запросе Flask-Login автоматически загружает пользователя через функцию `load_user` и предоставляет объект `current_user`. Проверка прав администратора выполняется декоратором `@admin_required`.

Модель `User` (`models.py`) наследуется от `UserMixin` и содержит методы поиска пользователя и проверки пароля.

Модуль `crud.py` отвечает за вставку и обновление записей о слушателях в рамках одной транзакции, охватывающей все связанные таблицы. Модуль `queries.py` формирует SQL-запросы с JOIN к тринадцати таблицам и динами-



ческой фильтрацией. Модуль `validation.py` проверяет корректность полей: формат СНИЛС, телефона, дат, согласованность дат и т.д.

### 3.4.1 Обработка запроса

На рисунке 3.4 приведена диаграмма последовательности, иллюстрирующая взаимодействие компонентов при обработке HTTP-запроса на добавление слушателя.

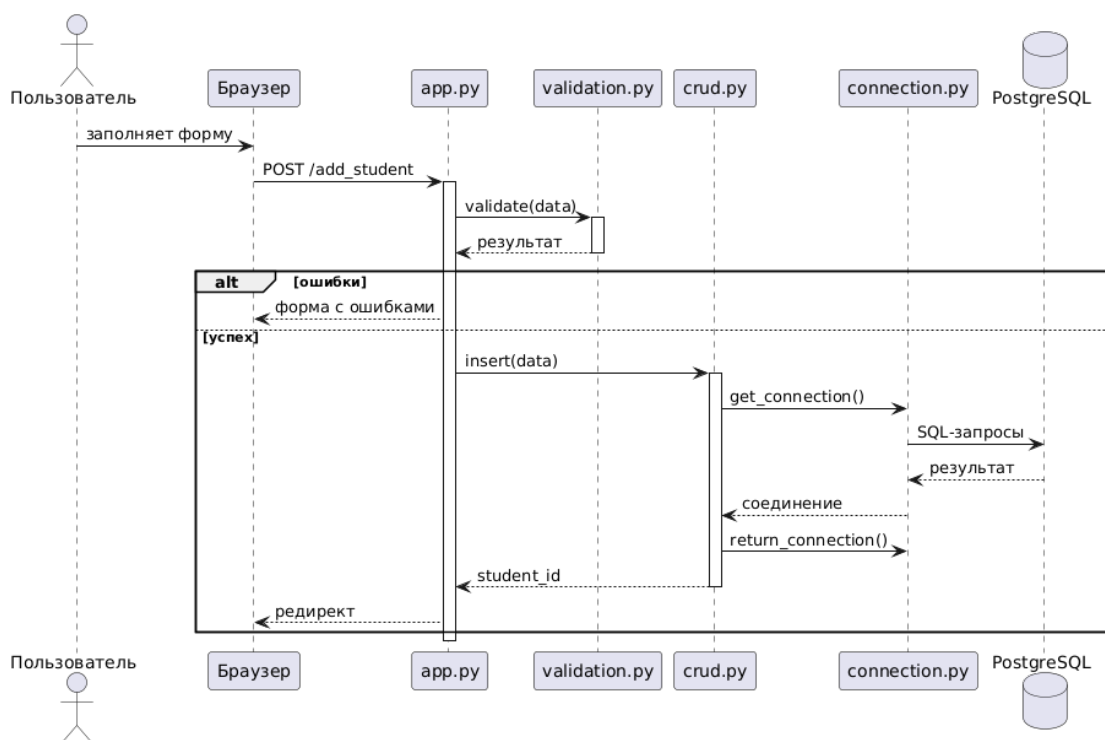


Рисунок 3.4 – Диаграмма последовательности обработки запроса

Пользователь через браузер отправляет POST-запрос с данными формы. Flask-приложение получает запрос, извлекает данные и передаёт их модулю валидации. При успешной проверке данные направляются в `crud.py`, который через `connection.py` выполняет серию SQL-команд. Результат возвращается Flask, который генерирует HTML-ответ с помощью Jinja2 и отправляет его браузеру.

### 3.4.2 Форматы данных

Обмен данными между браузером и веб-сервером ведётся по протоколу HTTP. Данные форм передаются в кодировке application/x-www-form-urlencoded. Результаты выборок из базы данных передаются в шаблоны Jinja2 как структуры Python (списки, словари) и преобразуются в HTML на стороне сервера. Импортируемые файлы имеют формат XLSX. Журнал событий ведётся в текстовом виде с временной меткой.

### 3.4.3 Программный интерфейс (API)

В таблице 3.1 приведены основные HTTP-запросы, обрабатываемые серверной частью. Все POST-запросы принимают данные HTML-форм в кодировке application/x-www-form-urlencoded.

Таблица 3.1 – Основные HTTP-запросы серверной части

| Маршрут              | Метод     | Входные данные                  | Результат                              |
|----------------------|-----------|---------------------------------|----------------------------------------|
| /                    | GET       | Параметры фильтрации в URL      | HTML-страница со списком слушателей    |
| /login               | GET, POST | username, password              | Вход в систему или страница с ошибкой  |
| /logout              | GET       | —                               | Выход из системы                       |
| /add_student         | GET, POST | Поля формы слушателя (45 полей) | Добавление записи или форма с ошибками |
| /edit_student/<id>   | GET, POST | id в URL + поля формы           | Обновление записи или форма с ошибками |
| /delete_student/<id> | GET       | id в URL                        | Удаление записи                        |

Продолжение таблицы 3.1

| Маршрут                  | Метод     | Входные данные                                   | Результат                                   |
|--------------------------|-----------|--------------------------------------------------|---------------------------------------------|
| /admin/users             | GET       | —                                                | Список пользователей                        |
| /admin/users/add         | GET, POST | username,<br>password,<br>is_admin               | Создание пользователя или форма с ошибкой   |
| /admin/users/edit/<id>   | GET, POST | id в URL +<br>username,<br>password,<br>is_admin | Обновление пользователя или форма с ошибкой |
| /admin/users/delete/<id> | GET       | id в URL                                         | Удаление пользователя                       |

В качестве примера обработки POST-запроса ниже показан маршрут добавления слушателя (/add\_student). Когда пользователь заполнил форму и нажал «Сохранить», браузер отправляет на сервер строку следующего вида (значения полей взяты для примера):

```
last_name=Иванов&first_name=Пётр&patronymic=Сергеевич&
birth_date=1990-01-01&gender=Муж&snils=123-456-789 00&
phone=+7-999-123-45-67&citizenship_code=643&
status_id=1&is_pedagogical=Нет&is_manager=Нет&
group_code=ПИ-23&program_type_id=1&program_name=Python&
department_name=Кафедра ПИ&learning_form_id=1&
uses_dot=t&duration_hours=72&start_date=2025-09-01&...
```

Flask автоматически преобразует строку от браузера в объект request.form (словарь Python). Вся дальнейшая обработка — валидация, сохранение, возврат в шаблон — работает уже с этим словарём, без разбора сырой строки. При успешной валидации программа добавляет нового

слушателя в базу данных и отправляет браузеру HTTP-редирект на главную страницу, которая загружается с обновлённым списком. Если валидация обнаружила ошибки, программа возвращает ту же страницу с формой и сообщениями об ошибках.

Аналогично устроены все POST-маршруты: `/edit_student/<id>` и `/admin/users/....` Маршруты с методом GET либо отдают готовую HTML-страницу, либо выполняют действие (удаление) и делают редирект.

### 3.5 Проект базы данных программно-информационной системы

#### 3.5.1 Компоненты базы данных

Компоненты, отвечающие за хранение данных и взаимодействие с СУБД, показаны на рисунке 3.5.

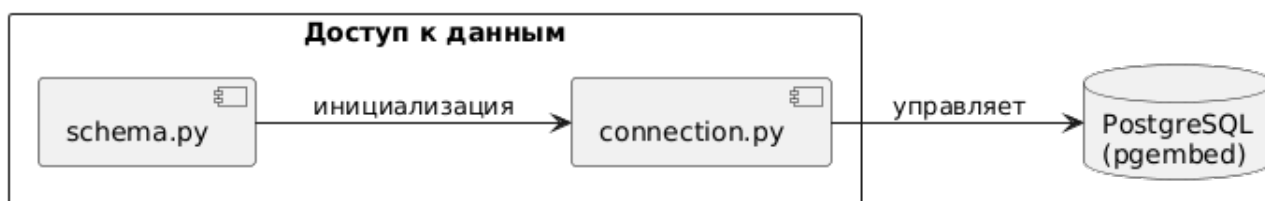


Рисунок 3.5 – Диаграмма компонентов базы данных

Внешней средой является СУБД PostgreSQL, встроенная через `pgembed`. Модуль `connection.py` управляет процессом PostgreSQL, создаёт пул соединений `ThreadedConnectionPool` (от 2 до 10 потокобезопасных соединений) и предоставляет методы `get_connection()` и `return_connection()` для остальных модулей.

Модуль `schema.py` вызывается однократно при первом запуске и выполняет создание всех таблиц, заполнение справочников и создание учётной записи администратора.

Физически данные хранятся в подкаталоге `./pgdata` каталога приложения. При повторных запусках инициализация пропускается.

### 3.5.2 ER-модель данных

На основе анализа предметной области были выделены основные сущности и связи между ними. ER-диаграмма (Entity-Relationship diagram, диаграмма «сущность-связь») представляет собой графическую схему, которая показывает, какие данные присутствуют в системе и как они связаны друг с другом. На рисунке 3.6 представлена ER-диаграмма базы данных.

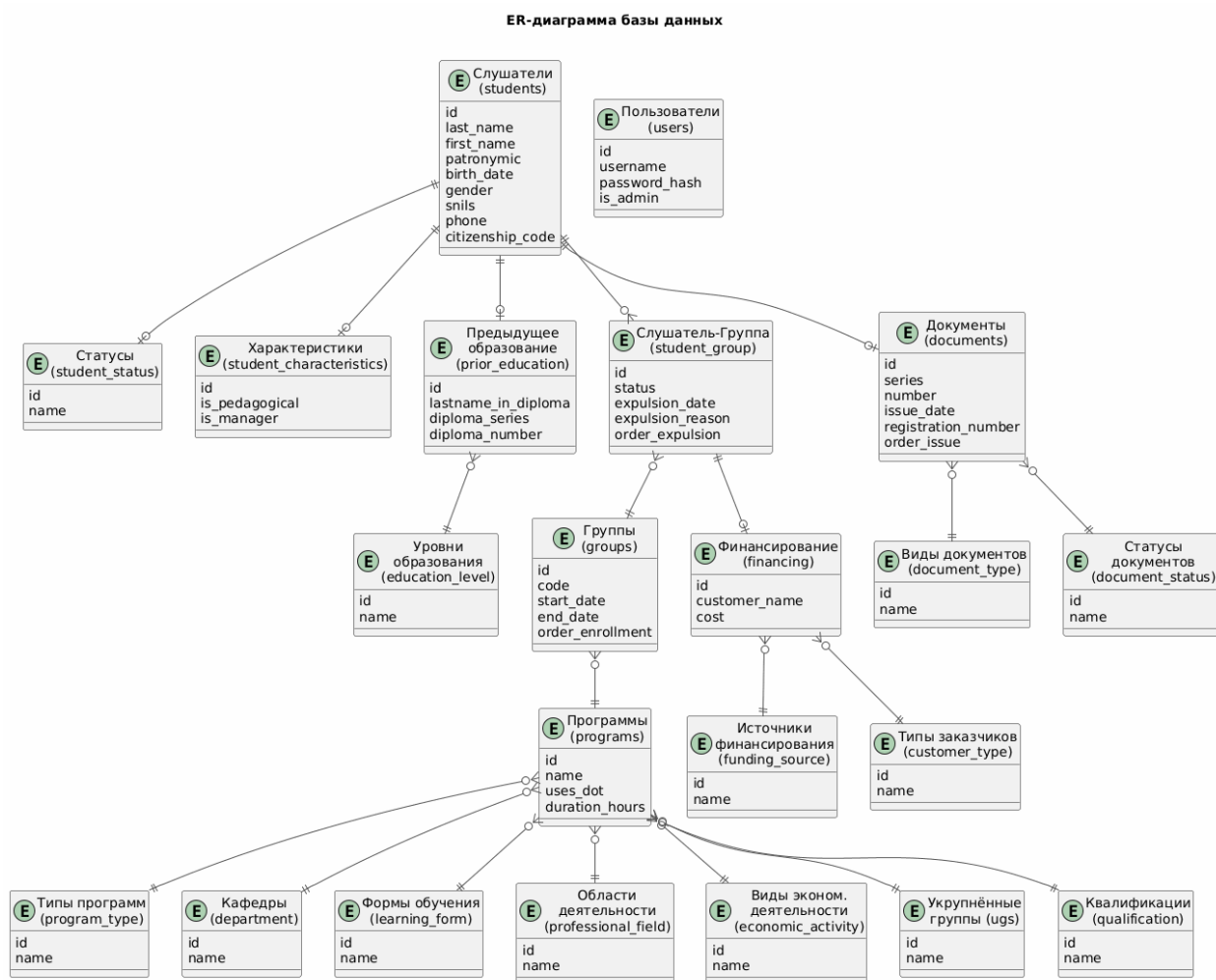


Рисунок 3.6 – ER-диаграмма базы данных

### 3.5.3 Нормализация

Схема базы данных спроектирована в соответствии с требованиями третьей нормальной формы. Все атрибуты атомарны, каждая таблица имеет суррогатный первичный ключ, ссылки на справочные таблицы реализованы



### 3.5.5 Описание структуры базы данных

В таблицах 3.2–3.23 приведено описание структуры таблиц базы данных.

Таблица 3.2 – Атрибуты таблицы «students» (Слушатели)

| Key Type | Optionality | Column name      | Data Type | Size       |
|----------|-------------|------------------|-----------|------------|
| PK       | *           | id               | SERIAL    |            |
|          | *           | last_name        | VARCHAR   | 100        |
|          | *           | first_name       | VARCHAR   | 100        |
|          |             | patronymic       | VARCHAR   | 100        |
|          | *           | birth_date       | DATE      | dd/mm/yyyy |
|          |             | gender           | VARCHAR   | 10         |
| UK       |             | snils            | VARCHAR   | 20         |
|          |             | phone            | VARCHAR   | 20         |
|          |             | citizenship_code | VARCHAR   | 10         |
| FK       |             | status_id        | INTEGER   |            |

Таблица 3.3 – Атрибуты таблицы «student\_status» (Статусы слушателей)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 200  |

Таблица 3.4 – Атрибуты таблицы «student\_characteristics» (Характеристики слушателя)

| Key Type | Optionality | Column name    | Data Type | Size |
|----------|-------------|----------------|-----------|------|
| PK       | *           | id             | SERIAL    |      |
| FK       | *           | student_id     | INTEGER   |      |
|          |             | is_pedagogical | VARCHAR   | 200  |

Продолжение таблицы 3.4

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
|          |             | is_manager  | VARCHAR   | 200  |

Таблица 3.5 – Атрибуты таблицы «prior\_education» (Предыдущее образование)

| Key Type | Optionality | Column name         | Data Type | Size |
|----------|-------------|---------------------|-----------|------|
| PK       | *           | id                  | SERIAL    |      |
| FK       | *           | student_id          | INTEGER   |      |
|          |             | lastname_in_diploma | VARCHAR   | 100  |
|          |             | diploma_series      | VARCHAR   | 50   |
|          |             | diploma_number      | VARCHAR   | 50   |
| FK       |             | education_level_id  | INTEGER   |      |

Таблица 3.6 – Атрибуты таблицы «education\_level» (Уровни образования)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 100  |

Таблица 3.7 – Атрибуты таблицы «student\_group» (Связь слушателей с группами)

| Key Type | Optionality | Column name    | Data Type | Size       |
|----------|-------------|----------------|-----------|------------|
| PK       | *           | id             | SERIAL    |            |
| FK       | *           | student_id     | INTEGER   |            |
| FK       | *           | group_id       | INTEGER   |            |
|          |             | status         | VARCHAR   | 50         |
|          |             | expulsion_date | DATE      | dd/mm/yyyy |



Продолжение таблицы 3.7

| Key Type | Optionality | Column name      | Data Type | Size |
|----------|-------------|------------------|-----------|------|
|          |             | expulsion_reason | TEXT      |      |
|          |             | order_expulsion  | VARCHAR   | 100  |

Таблица 3.8 – Атрибуты таблицы «groups» (Группы)

| Key Type | Optionality | Column name      | Data Type | Size       |
|----------|-------------|------------------|-----------|------------|
| PK       | *           | id               | SERIAL    |            |
|          | *           | code             | VARCHAR   | 50         |
| FK       |             | program_id       | INTEGER   |            |
|          |             | start_date       | DATE      | dd/mm/yyyy |
|          |             | end_date         | DATE      | dd/mm/yyyy |
|          |             | order_enrollment | VARCHAR   | 100        |

Таблица 3.9 – Атрибуты таблицы «programs» (Программы обучения)

| Key Type | Optionality | Column name           | Data Type | Size |
|----------|-------------|-----------------------|-----------|------|
| PK       | *           | id                    | SERIAL    |      |
|          | *           | name                  | VARCHAR   | 300  |
| FK       |             | program_type_id       | INTEGER   |      |
| FK       |             | department_id         | INTEGER   |      |
| FK       |             | learning_form_id      | INTEGER   |      |
|          |             | uses_dot              | BOOLEAN   |      |
|          |             | duration_hours        | INTEGER   |      |
| FK       |             | professional_field_id | INTEGER   |      |
| FK       |             | economic_activity_id  | INTEGER   |      |
| FK       |             | ugs_id                | INTEGER   |      |
| FK       |             | qualification_id      | INTEGER   |      |

Таблица 3.10 – Атрибуты таблицы «program\_type» (Типы программ)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 100  |

Таблица 3.11 – Атрибуты таблицы «department» (Кафедры)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 200  |

Таблица 3.12 – Атрибуты таблицы «learning\_form» (Формы обучения)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 50   |

Таблица 3.13 – Атрибуты таблицы «professional\_field» (Области профессиональной деятельности)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 200  |

Таблица 3.14 – Атрибуты таблицы «economic\_activity» (Виды экономической деятельности)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 300  |

Таблица 3.15 – Атрибуты таблицы «ugs» (Укрупнённые группы специальностей)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 300  |

Таблица 3.16 – Атрибуты таблицы «qualification» (Квалификации)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 300  |

Таблица 3.17 – Атрибуты таблицы «financing» (Финансирование)

| Key Type | Optionality | Column name       | Data Type | Size |
|----------|-------------|-------------------|-----------|------|
| PK       | *           | id                | SERIAL    |      |
| FK       | *           | student_group_id  | INTEGER   |      |
| FK       |             | funding_source_id | INTEGER   |      |
| FK       |             | customer_type_id  | INTEGER   |      |
|          |             | customer_name     | VARCHAR   | 300  |
|          |             | cost              | NUMERIC   | 12,2 |

Таблица 3.18 – Атрибуты таблицы «funding\_source» (Источники финансирования)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 100  |

Таблица 3.19 – Атрибуты таблицы «customer\_type» (Типы заказчиков)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 50   |

Таблица 3.20 – Атрибуты таблицы «documents» (Документы о квалификации)

| Key Type | Optionality | Column name         | Data Type | Size       |
|----------|-------------|---------------------|-----------|------------|
| PK       | *           | id                  | SERIAL    |            |
| FK       | *           | student_id          | INTEGER   |            |
| FK       |             | document_type_id    | INTEGER   |            |
| FK       |             | status_id           | INTEGER   |            |
|          |             | series              | VARCHAR   | 50         |
|          |             | number              | VARCHAR   | 50         |
|          |             | issue_date          | DATE      | dd/mm/yyyy |
|          |             | registration_number | VARCHAR   | 100        |
|          |             | order_issue         | VARCHAR   | 100        |

Таблица 3.21 – Атрибуты таблицы «document\_type» (Виды документов)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 200  |

Таблица 3.22 – Атрибуты таблицы «document\_status» (Статусы документов)

| Key Type | Optionality | Column name | Data Type | Size |
|----------|-------------|-------------|-----------|------|
| PK       | *           | id          | SERIAL    |      |
| UK       | *           | name        | VARCHAR   | 50   |

Таблица 3.23 – Атрибуты таблицы «users» (Пользователи системы)

| Key Type | Optionality | Column name   | Data Type | Size |
|----------|-------------|---------------|-----------|------|
| PK       | *           | id            | SERIAL    |      |
| UK       | *           | username      | VARCHAR   | 50   |
|          | *           | password_hash | VARCHAR   | 200  |
|          |             | is_admin      | BOOLEAN   |      |

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лутц, М. Изучаем Python. Том 1 / М. Лутц. – 5-е изд. – Санкт-Петербург : Питер, 2019. – 832 с. – ISBN 978-5-907144-52-1. – Текст : непосредственный.
2. Рамальо, Л. Python. К вершинам мастерства / Л. Рамальо. – Москва : ДМК Пресс, 2017. – 768 с. – ISBN 978-5-97060-384-3. – Текст : непосредственный.
3. Python Documentation : сайт. – URL: <https://docs.python.org/3/> (дата обращения: 24.04.2026). – Текст : электронный.
4. PEP 8 – Style Guide for Python Code : сайт. – URL: <https://peps.python.org/pep-0008/> (дата обращения: 24.04.2026). – Текст : электронный.
5. Параллельное программирование в Python (threading) : сайт. – URL: <https://docs.python.org/3/library/threading.html> (дата обращения: 24.04.2026). – Текст : электронный.
6. Logging HOWTO : сайт. – URL: <https://docs.python.org/3/howto/logging.html> (дата обращения: 24.04.2026). – Текст : электронный.
7. Гринберг, М. Разработка веб-приложений с использованием Flask на языке Python / М. Гринберг. – Москва : ДМК Пресс, 2014. – 272 с. – ISBN 978-5-97060-206-5. – Текст : электронный.
8. Flask Documentation : сайт. – URL: <https://flask.palletsprojects.com/> (дата обращения: 24.04.2026). – Текст : электронный.
9. Flask-Login Documentation : сайт. – URL: <https://flask-login.readthedocs.io/> (дата обращения: 24.04.2026). – Текст : электронный.
10. Jinja2 Documentation : сайт. – URL: <https://jinja.palletsprojects.com/> (дата обращения: 24.04.2026). – Текст : электронный.

11. Werkzeug Documentation : сайт. – URL: <https://werkzeug.palletsprojects.com/> (дата обращения: 24.04.2026). – Текст : электронный.

12. Дейт, К. Дж. Введение в системы баз данных / К. Дж. Дейт. – 8-е изд. – Москва : Вильямс, 2005. – 1328 с. – ISBN 978-5-8459-0788-2. – Текст : непосредственный.

13. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. – 3-е изд. – Москва : Вильямс, 2003. – 1440 с. – ISBN 978-5-8459-0527-7. – Текст : непосредственный.

14. Грофф, Дж. Р. SQL: полное руководство / Дж. Р. Грофф, П. Н. Вайнберг, Э. Дж. Оппель. – 3-е изд. – Санкт-Петербург : Диалектика, 2020. – 960 с. – ISBN 978-5-907144-26-5. – Текст : непосредственный.

15. Чен, П. Модель «сущность-связь» – шаг к единому представлению данных / П. Чен. – Москва : Мир, 1983. – Текст : непосредственный.

16. Кент, У. Простое руководство по пяти нормальным формам в теории реляционных баз данных / У. Кент. – Москва : Мир, 1989. – Текст : непосредственный.

17. Хардер, Т. Принципы транзакционной обработки баз данных / Т. Хардер, А. Ройтер. – Текст : непосредственный // Системы управления базами данных. – 2004. – № 2. – С. 56-71.

18. Моргунов, Е. П. PostgreSQL. Основы языка SQL / Е. П. Моргунов. – Санкт-Петербург : БХВ-Петербург, 2022. – 336 с. – ISBN 978-5-9775-3960-9. – Текст : непосредственный.

19. PostgreSQL Documentation : сайт. – URL: <https://www.postgresql.org/docs/> (дата обращения: 24.04.2026). – Текст : электронный.

20. Psycopg – PostgreSQL adapter for Python : сайт. – URL: <https://www.psycopg.org/docs/> (дата обращения: 24.04.2026). – Текст : электронный.

21. PyPgEmbed – Embedded PostgreSQL for Python : сайт. – URL: <https://github.com/Simulmedia/pyembedpg> (дата обращения: 24.04.2026). – Текст : электронный.

22. Управление пулом соединений PostgreSQL : сайт. – URL: <https://www.postgresql.org/docs/> (дата обращения: 24.04.2026). – Текст : электронный.

23. Tkinter Documentation : сайт. – URL: <https://docs.python.org/3/library/tkinter.html> (дата обращения: 24.04.2026). – Текст : электронный.

24. Ткаченко, А. Tkinter. Программирование GUI на Python / А. Ткаченко. – Текст : электронный // Metanit. – URL: <https://metanit.com/python/tkinter/> (дата обращения: 24.04.2026).

25. Дзекетт, Д. HTML и CSS. Разработка и создание веб-сайтов / Д. Дзекетт. – Москва : Эксмо, 2014. – 480 с. – ISBN 978-5-699-64193-2. – Текст : непосредственный.

26. HTML и CSS. Руководство для начинающих : сайт. – URL: <https://developer.mozilla.org/ru/docs/Learn/HTML> (дата обращения: 24.04.2026). – Текст : электронный.

27. HTTP (HyperText Transfer Protocol) : сайт. – URL: <https://developer.mozilla.org/ru/docs/Web/HTTP> (дата обращения: 24.04.2026). – Текст : электронный.

28. Pandas – Python Data Analysis Library : сайт. – URL: <https://pandas.pydata.org/docs/> (дата обращения: 24.04.2026). – Текст : электронный.

29. Стандарт открытого формата Office Open XML : сайт. – URL: <https://www.ecma-international.org/publications-and-standards/standards/ecma-376/> (дата обращения: 24.04.2026). – Текст : электронный.

30. Буч, Г. Введение в UML от создателей языка / Г. Буч, И. Якобсон, Д. Рамбо. – Москва : ДМК Пресс, 2015. – 498 с. – ISBN 978-5-457-43379-3. – Текст : непосредственный.



31. Рамбо, Дж. UML 2.0. Объектно-ориентированное моделирование и разработка / Дж. Рамбо, М. Блаха. – 2-е изд. – Санкт-Петербург : Питер, 2021. – 542 с. – ISBN 978-5-4461-9428-5. – Текст : непосредственный.
32. Филд, Р. Т. Архитектурный стиль REST и проектирование сетевых приложений / Р. Т. Филд. – Текст : электронный. – URL: [https://www.ics.uci.edu/fielding/pubs/dissertation/rest\\_arch\\_style.htm](https://www.ics.uci.edu/fielding/pubs/dissertation/rest_arch_style.htm) (дата обращения: 24.04.2026).
33. Мартин, Р. Чистый код: создание, анализ и рефакторинг / Р. Мартин. – Санкт-Петербург : Питер, 2013. – 464 с. – ISBN 978-5-496-00487-9. – Текст : непосредственный.
34. Майерс, Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер. – 3-е изд. – Москва : Вильямс, 2012. – 272 с. – ISBN 978-5-8459-1796-6. – Текст : непосредственный.
35. PyCharm Documentation : сайт. – URL: <https://www.jetbrains.com/pycharm/documentation/> (дата обращения: 24.04.2026). – Текст : электронный.
36. Чакон, С. Git для профессионального программиста / С. Чакон, Б. Штрауб. – Санкт-Петербург : Питер, 2016. – 496 с. – ISBN 978-5-496-01763-3. – Текст : непосредственный.
37. pgAdmin – PostgreSQL Tools : сайт. – URL: <https://www.pgadmin.org/docs/> (дата обращения: 24.04.2026). – Текст : электронный.
38. Котельников, И. А. LaTeX по-русски / И. А. Котельников, П. З. Чебо-таев. – Санкт-Петербург : БХВ-Петербург, 2004. – 498 с. – ISBN 5-87550-195-2. – Текст : непосредственный.
39. Гринберг, М. Разработка веб-приложений с использованием Flask на языке Python / М. Гринберг. – 2-е изд. – Санкт-Петербург : Питер, 2019. – 320 с. – ISBN 978-5-4461-1231-0. – Текст : непосредственный.
40. Мур, А. Д. Python GUI Programming with Tkinter / А. Д. Мур. – 2-е изд. – Packt Publishing, 2021. – 528 с. – ISBN 978-1-80181-592-5. – Текст : непосредственный.

41. Васильев, А. В. PostgreSQL 14. Защита, настройка и оптимизация / А. В. Васильев. – Санкт-Петербург : БХВ-Петербург, 2023. – 448 с. – ISBN 978-5-9775-1741-6. – Текст : непосредственный.
42. Бейли, Л. Изучаем SQL / Л. Бейли. – Санкт-Петербург : Питер, 2022. – 384 с. – ISBN 978-5-4461-0984-8. – Текст : непосредственный.
43. Маккинни, У. Python и анализ данных / У. Маккинни. – 3-е изд. – Москва : ДМК Пресс, 2023. – 536 с. – ISBN 978-5-93700-174-0. – Текст : непосредственный.
44. Оккен, Б. pytest. Быстрое тестирование приложений на Python / Б. Оккен. – Санкт-Петербург : Питер, 2020. – 320 с. – ISBN 978-5-4461-1265-4. – Текст : непосредственный.
45. Персиваль, Г. Python. Разработка на основе тестирования / Г. Персиваль. – Москва : ДМК Пресс, 2018. – 622 с. – ISBN 978-5-97060-594-3. – Текст : непосредственный.
46. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург : Питер, 2018. – 352 с. – ISBN 978-5-4461-0772-8. – Текст : непосредственный.
47. Ганди, Р. Head First. Git / Р. Ганди. – Санкт-Петербург : БХВ-Петербург, 2023. – 464 с. – ISBN 978-5-9775-1777-5. – Текст : непосредственный.
48. Фрейн, Б. HTML5 и CSS3. Разработка современных веб-сайтов / Б. Фрейн. – 2-е изд. – Санкт-Петербург : Питер, 2022. – 384 с. – ISBN 978-5-4461-0526-2. – Текст : непосредственный.
49. TkDocs – Tk Tutorial : сайт. – URL: <https://tkdocs.com/> (дата обращения: 24.04.2026). – Текст : электронный.
50. SQLAlchemy Documentation : сайт. – URL: <https://docs.sqlalchemy.org/> (дата обращения: 24.04.2026). – Текст : электронный.
51. Alembic – Database Migrations for SQLAlchemy : сайт. – URL: <https://alembic.sqlalchemy.org/> (дата обращения: 24.04.2026). – Текст : электронный.

52. Bootstrap Documentation : сайт. – URL: <https://getbootstrap.com/docs/5.3/> (дата обращения: 24.04.2026). – Текст : электронный.

53. Мартин, Р. The Clean Architecture (статья) : сайт. – URL: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html> (дата обращения: 24.04.2026). – Текст : электронный.